Cybersecurity Lab 1 · ELTE Faculty of Informatics · Autumn 2026

Comparative study of post-quantum key-encapsulation mechanisms

Part 1: performance (E1) and correctness (E4)

Hamdan Al-Jabri

11 October 2026

At a glance

69/69
correctness checks pass on both machines
KATs, NIST ACVP (FIPS 203), random round trips, ref↔opt, liboqs↔OpenSSL; 1,332 vectors each

2.2× / 3.5×
ML-KEM-768 is cheaper than X25519
compute for a full key exchange vs classical X25519 ECDH (M4 / i7)

1.6× / 1.7×
the deployed hybrid vs X25519
X25519MLKEM768 costs exactly its two parts added together

43–841×
the conservative KEMs vs X25519
HQC, FrodoKEM and Classic McEliece at level 1 (M4)

25–16,000×
more bytes on the wire than X25519
ML-KEM-768: 2,272 B vs 64 B; Classic McEliece: up to 1 MB public keys

up to 1,400×
speed-up from AVX2 code
McEliece decaps on the i7; NEON exists only for ML-KEM, so the M4 gains 1.4–1.7× there

Measurements: Apple M4 (ARM64, Docker Desktop) and Intel Core i7-1185G7 (x86-64, WSL2), liboqs 0.16.0 and OpenSSL 3.5.8.
All figures and tables are generated by `analysis/part1_figures.py` from `results/raw`.
Code and data: `github.com/hsh1994-ux/pq-kem-study`.

Contents

1 Why these four KEMs **2**

2 What is a KEM? **2**

3 From Diffie–Hellman to a KEM **3**
3.1 X25519 Diffie–Hellman . . . 3
3.2 Two ways to put X25519 next to the KEMs . . . 4
3.3 The hybrid X25519MLKEM768 . . . 4

4 Experimental setup and our choices **4**
4.1 Machines . . . 4
4.2 Implementations and builds . . . 5
4.3 Measurement method . . . 5

5 E4: correctness **5**

6 E1: performance **7**
6.1 Sizes: what goes over the wire . . . 7
6.2 The full exchange . . . 8
6.3 Per operation . . . 9
6.4 The cost/size trade-off . . . 10
6.5 Security level scaling . . . 10
6.6 Optimised vs. portable code . . . 11
6.7 ARM64 vs. x86-64 . . . 12
6.8 The classical baseline and the hybrid . . . 13
6.9 Memory: stack use . . . 14
6.10 Validation against published results . . . 14

7 Findings, limitations and next steps **15**

A Full E1 results **15**

B Reproducing this report **16**

References **16**

Scope of Part 1

- **RQ1 (cost).** How much computation, memory and bandwidth does each KEM need, compared with classical X25519? Answered by **E1**.
- **RQ4 (correctness).** Do the implementations we measure compute the right thing, on both architectures and across libraries? Answered by **E4**.
- E2 (TLS handshakes) and E3 (timing side channels) are reported in Part 2.

1 Why these four KEMs

Post-quantum KEMs fall into a few families, depending on the hard problem their security rests on. We picked one representative from each corner of a 2×2 map: **lattice vs. code-based**, and **structured vs. unstructured** (Figure 1).

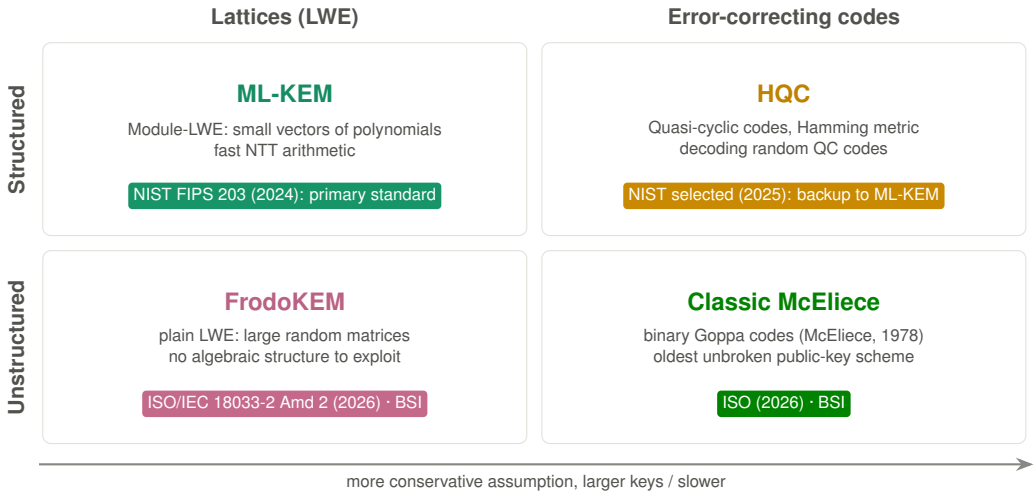


Figure 1. The four KEMs cover both main post-quantum families, each with and without algebraic structure.

KEM	Hard problem	Status (Oct 2026)	Why it is in the study
ML-KEM	Module Learning With Errors	FIPS 203 (Aug 2024); in TLS, browsers, OpenSSL 3.5	The standard everyone will deploy: the reference point.
HQC	Decoding quasi-cyclic codes	Selected by NIST (Mar 2025); draft standard pending	NIST's backup if lattices break: different maths, same role.
FrodoKEM	Plain LWE (unstructured)	ISO/IEC 18033-2 Amd 2 (2026); recommended by BSI, NLNCSA	Same family as ML-KEM *without* structure: shows what structure buys.
Classic McEliece	Decoding binary Goppa codes	ISO (2026); recommended by BSI; not selected by NIST	Most conservative choice; extreme size trade-off (tiny ciphertext, huge key).

Table 1. The four KEMs. All four are in liboqs with a portable and an optimised implementation, at NIST levels 1, 3 and 5.

Our choices

- **Diversity of assumptions:** if one hard problem falls, the comparison still says what the alternatives cost.
- **Structure vs. no structure** inside each family (ML-KEM/FrodoKEM, HQC/McEliece) isolates the price of conservatism.
- **Standardisation relevance:** every KEM is standardised or selected by NIST or ISO, or recommended by a national agency.
- **Three security levels each** (NIST 1, 3, 5 ≈ AES-128/192/256): 12 KEMs in total.
- **Two baselines:** classical **X25519** (what TLS uses today) and the deployed hybrid **X25519MLKEM768**.

2 What is a KEM?

A **key-encapsulation mechanism** lets two parties agree on a shared secret key over an insecure channel, using public-key cryptography. It is three algorithms:

Definition: KEM = (KeyGen, Encaps, Decaps)

$\mathsf{KeyGen}() \to (pk, sk)$	receiver (client)	makes a key pair; pk is sent
$\mathsf{Encaps}(pk) \to (ct, K)$	sender (server)	picks a fresh secret K and "wraps" it into ct
$\mathsf{Decaps}(sk, ct) \to K$	receiver (client)	unwraps ct with sk and gets the same K

Correctness: $\mathsf{Decaps}(sk, ct) = K$ (except with negligible probability). **Security (IND-CCA2):** seeing pk and ct, and even decapsulating other ciphertexts, an attacker cannot tell K from random.

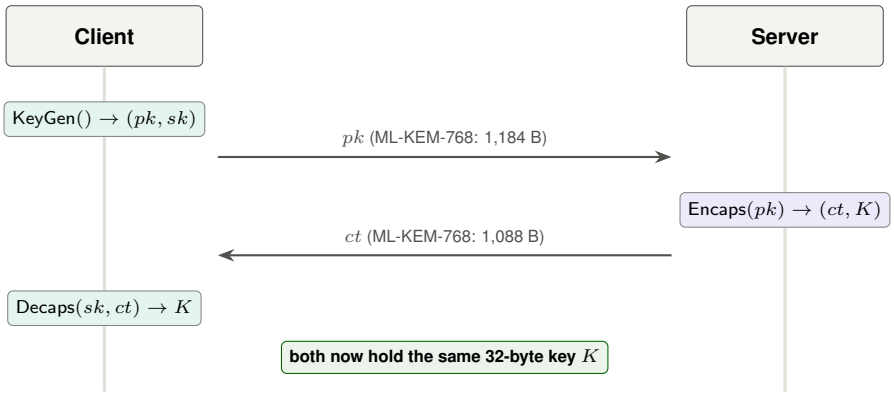


Figure 2. One key exchange with a KEM: one message each way. E1 times each of the three boxes.

How post-quantum KEMs get CCA security. ML-KEM, HQC and FrodoKEM start from a public-key encryption scheme and apply a **Fujisaki–Okamoto-style transform**: Decaps decrypts, *re-encrypts* and compares, returning a pseudo-random key on mismatch (implicit rejection). Classic McEliece uses its own Niederreiter-based construction, also with implicit rejection. The re-encryption is why **Decaps is usually the most expensive operation** for the lattice and HQC schemes (Section 6.3).

Why KEMs and not Diffie–Hellman? Post-quantum cryptography has no efficient drop-in replacement for Diffie–Hellman's non-interactive "combine my secret with your public value". A KEM gives the same result, a shared key after one round trip, with a slightly different interface. NIST therefore standardised KEMs, and TLS 1.3 can carry them in its existing key-share messages.

3 From Diffie–Hellman to a KEM

3.1 X25519 Diffie–Hellman

X25519 (RFC 7748) works on Curve25519. A private key is a random 32-byte scalar a; the public key is $A = a \cdot G$. With the peer's public key B, both sides compute the same point $a \cdot B = b \cdot A = ab \cdot G$. TLS 1.3 uses it **ephemerally** (ECDHE): fresh key pairs on both sides for every handshake.

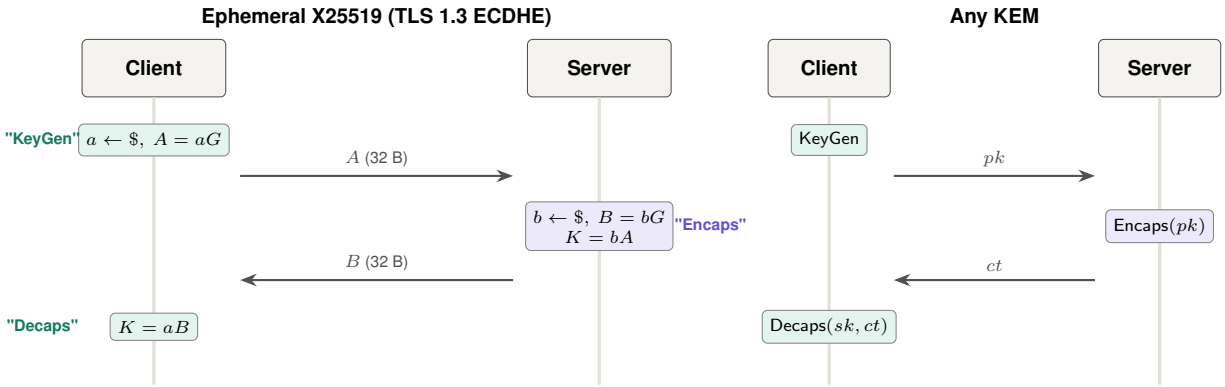


Figure 3. Ephemeral Diffie–Hellman already has the shape of a KEM: the client's key pair plays KeyGen, the server's key pair plus DH plays Encaps, and the client's DH plays Decaps.

3.2 Two ways to put X25519 next to the KEMs

KEM step	**X25519-ECDH** (plain, as in TLS 1.3) — *our classical baseline*	**X25519-DHKEM** (RFC 9180, HPKE)
KeyGen	$a \leftarrow \$,\ A = aG$	same
Encaps(A)	$b \leftarrow \$,\ B = bG,\ K = bA$; send B	$(e, E) \leftarrow$ DeriveKeyPair; $K = \mathsf{ExtractAndExpand}(eA,\ E\|\|A)$; send $ct = E$
Decaps(a, B)	$K = aB$	$K = \mathsf{ExtractAndExpand}(aE,\ E\|\|A)$
Cost	2 key pairs + 2 DH per exchange	same + HKDF on both sides
Sizes	pk = 32 B, "ct" = 32 B, K = 32 B	same

Table 2. X25519 measured as a KEM. We time both; the plain ECDH version is the fair baseline because TLS 1.3 does not use the HKDF wrapper (its own key schedule hashes K later).

3.3 The hybrid X25519MLKEM768

Deployed TLS combines both, so that the key stays safe if *either* X25519 (quantum computers) or ML-KEM (new cryptanalysis) is broken:

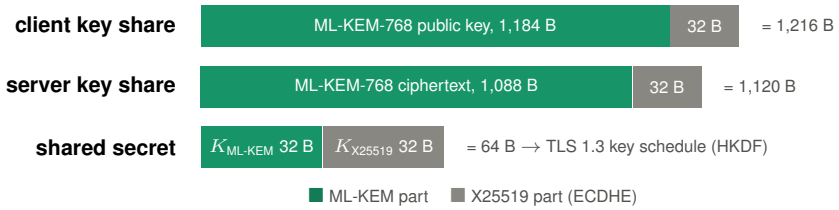


Figure 4. X25519MLKEM768 (IETF `draft-ietf-tls-ecdhe-mlkem`) concatenates both key shares and both secrets. The sizes are exactly those measured in E1.

4 Experimental setup and our choices

4.1 Machines

	Apple M4 (ARM64)	Intel i7-1185G7 (x86-64)
CPU	Apple M4	Intel i7-1185G7
cores	10	8
environment	Docker Desktop (Linux VM)	WSL2 (Ubuntu 24.04)
kernel	7.0.14-linuxkit	6.18.40.1-microsoft-standard-WSL2
compiler	GCC 13.3.0 (Ubuntu 24.04)	GCC 13.3.0 (Ubuntu 24.04)
liboqs	0.16.0	0.16.0
OpenSSL	3.5.8	3.5.8
timer (MHz)	24.000	1857.904
core clock at start (GHz)	4.395	3.890
timed calls per op	1000	1000
McEliece keygens	100	100
cooldown (s)	30	30
warm-up (ms)	200	200
code commit	124a4d6	e144cb6

Table 3. The two machines, read from the metadata header of each results file. Both run the same Ubuntu 24.04 userland, compiler and library versions; only the CPU differs.

4.2 Implementations and builds

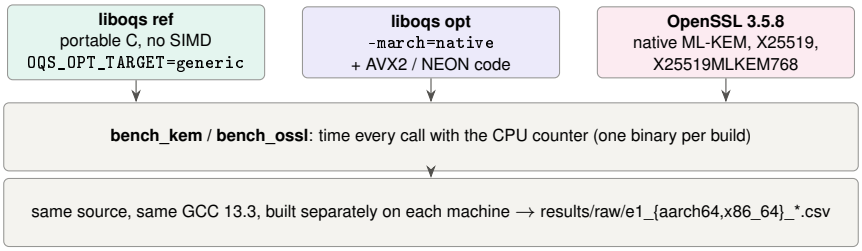


Figure 5. Three builds per machine. ref vs opt isolates what hand-optimised code adds; OpenSSL is the implementation that TLS deployments actually run.

KEM family	x86-64: AVX2 code	ARM64: NEON code	consequence
ML-KEM (mlkem-native)	✓	✓	opt faster on both
FrodoKEM	✓	–	opt = ref on M4
Classic McEliece	✓	–	opt = ref on M4
HQC	–	–	opt = ref (only compiler gains)

Table 4. Optimised code available in liboqs 0.16.0. This table explains most of the cross-machine differences in Section 6.7.

4.3 Measurement method

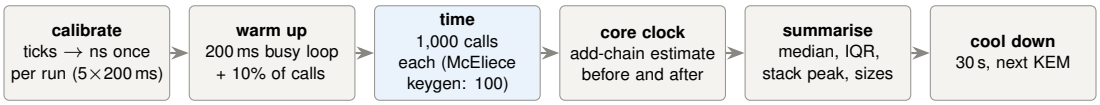


Figure 6. E1 pipeline, repeated for every KEM, build and operation (90 rows per machine). The process is pinned to one core.

Choice	Why
Time every call individually, report the **median** and IQR	Robust to interrupts and the occasional slow call; the IQR shows stability.
Hardware counter (TSC on x86, 24 MHz CNTVCT on ARM)	Readable inside Docker and WSL2, where the real cycle counter is not; converted to ns once per run.
200 ms warm-up	Measured: the M4 needs ∼90 ms to ramp from 2.0 to 4.4 GHz after idling.
Core-clock estimate (chain of dependent adds)	Lets us report **core cycles**, comparable with papers, despite turbo and AVX frequency changes.
30 s cooldown between KEMs	The MacBook Air is fanless: without pauses later KEMs would run on a hotter, slower chip.
100 McEliece keygens	Its key generation retries until a random matrix works; 20 samples gave unstable medians.
ref + opt builds of the same library	Separates algorithm cost from implementation effort.
Two architectures, same software	ARM64 vs x86-64 with identical code, compiler and versions.
Full exchange = keygen + encaps + decaps	One complete key agreement, comparable across KEMs and with ECDH.

Table 5. Methodological choices and their justification.

5 E4: correctness

Before comparing speed, we checked that every implementation computes the right answer.

Check	What it proves	Applies to
KAT	Deterministic output hashed and compared with the reference hash shipped by liboqs	all 12 KEMs, both builds
NIST ACVP	keyGen, encapsulation and decapsulation vectors from NIST's FIPS 203 validation suite	ML-KEM (60 per level)
Round trips	Random keygen → encaps → decaps gives the same key (50, McEliece 3)	all, both builds
Cross-build	Key from one build, encapsulate with the other, decapsulate with the first (both ways)	all
Interop	liboqs ↔ OpenSSL 3.5 native ML-KEM, both directions	ML-KEM

Table 6. The five E4 checks.

	KAT ref	KAT opt	round trips ref	round trips opt	NIST ACVP ref	NIST ACVP opt	cross-build ref↔opt	OpenSSL interop
ML-KEM-512	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	✓ 60 ✓ 60	✓ 60 ✓ 60	✓ 2 ✓ 2	✓ 2 ✓ 2
ML-KEM-768	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	✓ 60 ✓ 60	✓ 60 ✓ 60	✓ 2 ✓ 2	✓ 2 ✓ 2
ML-KEM-1024	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	✓ 60 ✓ 60	✓ 60 ✓ 60	✓ 2 ✓ 2	✓ 2 ✓ 2
HQC-1	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	–	–	✓ 2 ✓ 2	–
HQC-3	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	–	–	✓ 2 ✓ 2	–
HQC-5	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	–	–	✓ 2 ✓ 2	–
Frodo-640	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	–	–	✓ 2 ✓ 2	–
Frodo-976	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	–	–	✓ 2 ✓ 2	–
Frodo-1344	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 50 ✓ 50	✓ 50 ✓ 50	–	–	✓ 2 ✓ 2	–
McEliece-348864	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 3 ✓ 3	✓ 3 ✓ 3	–	–	✓ 2 ✓ 2	–
McEliece-460896	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 3 ✓ 3	✓ 3 ✓ 3	–	–	✓ 2 ✓ 2	–
McEliece-6688128	✓ 1 ✓ 1	✓ 1 ✓ 1	✓ 3 ✓ 3	✓ 3 ✓ 3	–	–	✓ 2 ✓ 2	–

each cell: M4 result, then i7 result (✓ n = n vectors or round trips passed); – = not applicable
M4 69/69 checks, 1,332 vectors · i7 69/69 checks, 1,332 vectors

Figure 7. E4 results. Every applicable check passes on both machines, 1,332 vectors and round trips each.

Takeaways

- All 69 checks pass on ARM64 and x86-64: **the optimised AVX2/NEON code computes exactly what the portable code does**, so E1 compares like with like.
- ML-KEM in liboqs matches NIST's official FIPS 203 vectors and interoperates with OpenSSL's independent implementation.
- Limitation: ACVP's key-check groups (FIPS 203 input validation) are not exposed by the liboqs API and are skipped, as in liboqs' own test.

6 E1: performance

Best build below means liboqs `opt` for the post-quantum KEMs and OpenSSL for X25519 and the hybrid. Times are medians per call; a full exchange is the sum of the three medians.

6.1 Sizes: what goes over the wire

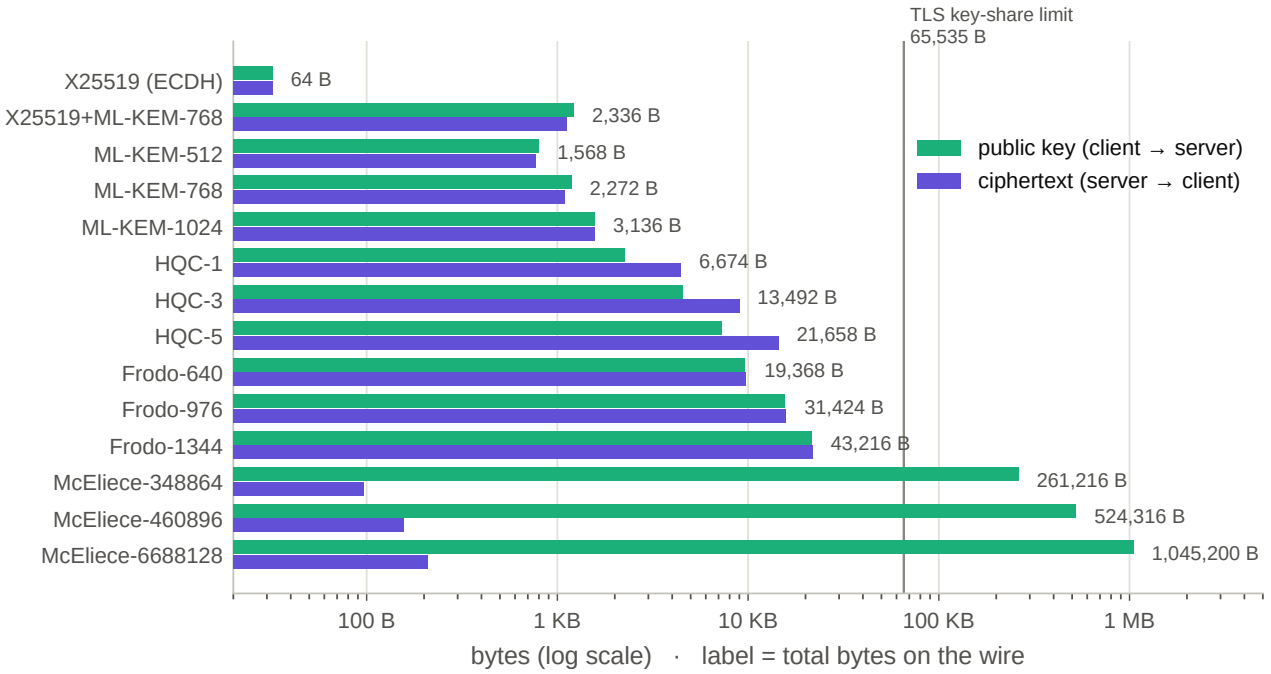


Figure 8. Public key and ciphertext per KEM (identical on both machines). McEliece's public key does not fit in a TLS key share.

KEM	level	public key	secret key	ciphertext	shared secret	on wire
X25519 (ECDH)	–	32	32	32	32	64
ML-KEM-512	1	800	1,632	768	32	1,568
ML-KEM-768	3	1,184	2,400	1,088	32	2,272
ML-KEM-1024	5	1,568	3,168	1,568	32	3,136
HQC-1	1	2,241	2,321	4,433	32	6,674
HQC-3	3	4,514	4,602	8,978	32	13,492
HQC-5	5	7,237	7,333	14,421	32	21,658
Frodo-640	1	9,616	19,888	9,752	16	19,368
Frodo-976	3	15,632	31,296	15,792	24	31,424
Frodo-1344	5	21,520	43,088	21,696	32	43,216
McEliece-348864	1	261,120	6,492	96	32	261,216
McEliece-460896	3	524,160	13,608	156	32	524,316
McEliece-6688128	5	1,044,992	13,932	208	32	1,045,200
X25519+ML-KEM-768	–	1,216	–	1,120	64	2,336

Table 7. Sizes in bytes, as reported by the implementations; they match the specifications exactly.

Takeaways

- ML-KEM: **25–49× X25519's 64 bytes** (35× at level 3), still only 1.5–3 KB. HQC is 4–7× ML-KEM, FrodoKEM 12–14×.
- Classic McEliece is the opposite trade: **the smallest ciphertext (96–208 B) but 0.26–1 MB public keys**, beyond the 65,535-byte TLS limit.

6.2 The full exchange

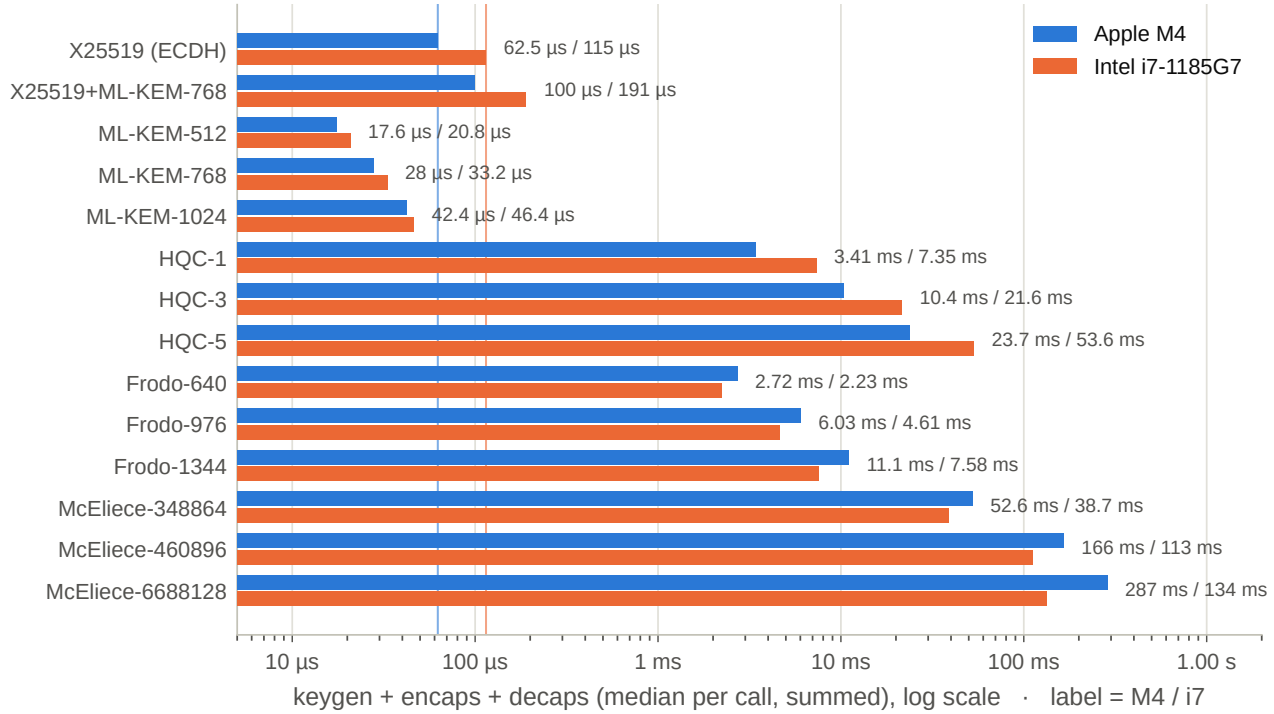


Figure 9. Compute for one complete key exchange, best build. The thin vertical lines mark X25519 ECDH on each machine.

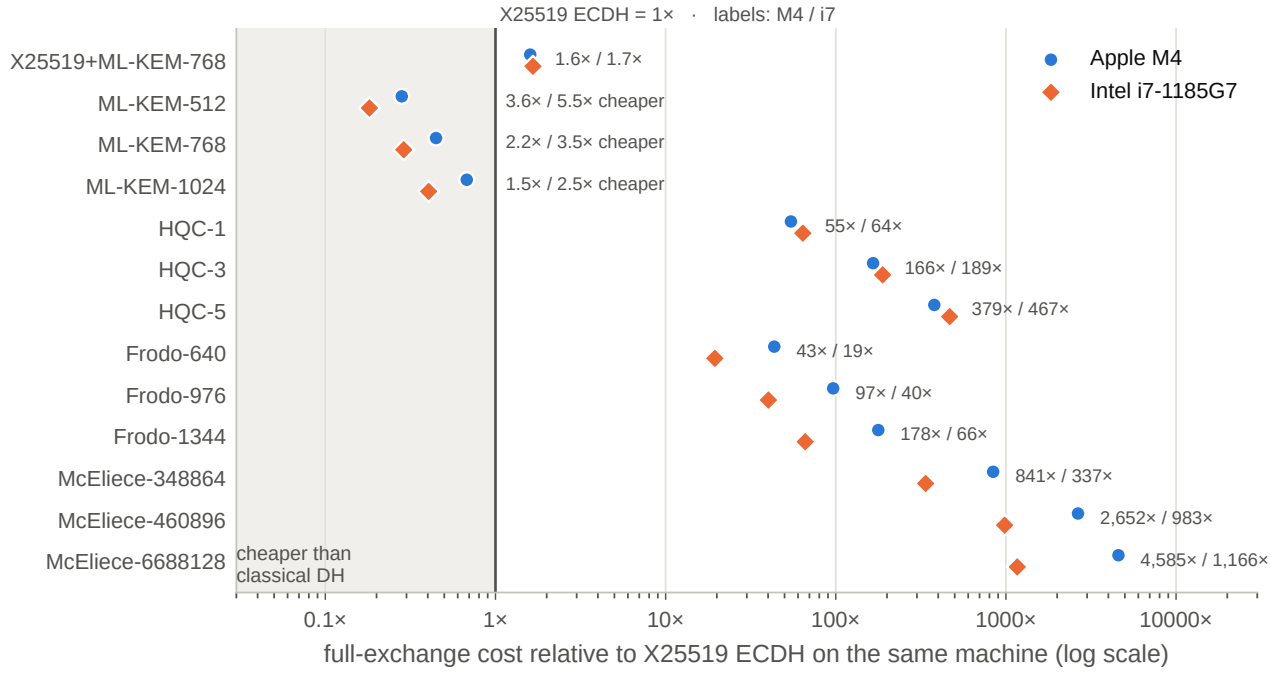


Figure 10. The same, relative to X25519 ECDH on the same machine (shaded = cheaper than classical DH).

Takeaways

- **ML-KEM is cheaper than classical X25519** at every level: level 3 is 2.2× (M4) and 3.5× (i7) cheaper; level 1 up to 5.5×.
- The **hybrid costs 1.6–1.7× X25519**: adding post-quantum security to today's TLS is cheap in compute.
- HQC (55–64× at level 1), FrodoKEM (19–43×) and McEliece (337–841×) cost one to three orders of magnitude more.

6.3 Per operation

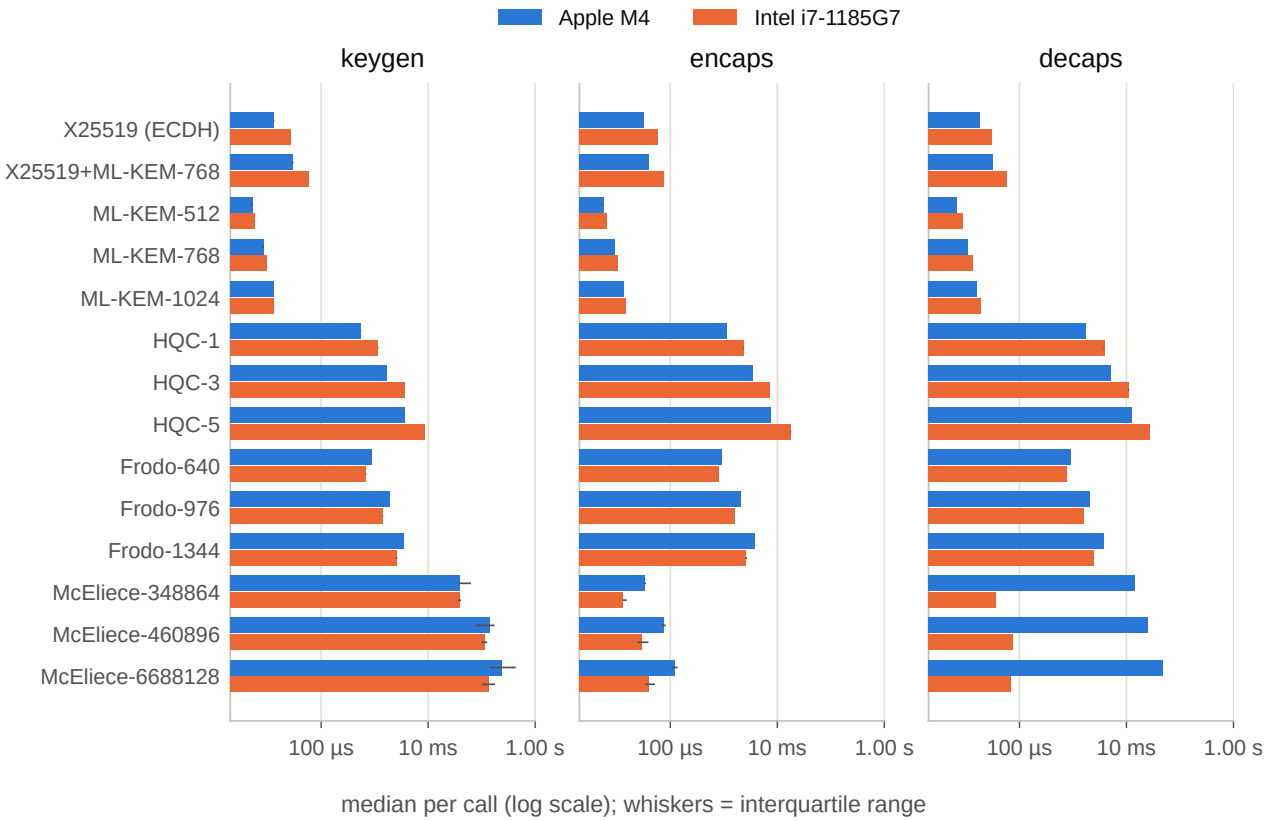


Figure 11. Median per call for each operation, best build, both machines.

- **ML-KEM, HQC:** decaps > encaps > keygen, because of the re-encryption check. HQC follows the 1:2:3 ratio of its specification.
- **FrodoKEM:** all three about equal, since generating the large matrix from a seed dominates every operation.
- **Classic McEliece:** keygen 39–239 ms (it retries until a random matrix gives a valid key), encaps 13–121 μs (very fast), decaps 37–73 μs with AVX2 but 14–48 ms without. The work sits with whoever makes the key.
- **X25519:** encaps $\approx$ keygen + decaps, since the server does both a key pair and a DH.

6.4 The cost/size trade-off

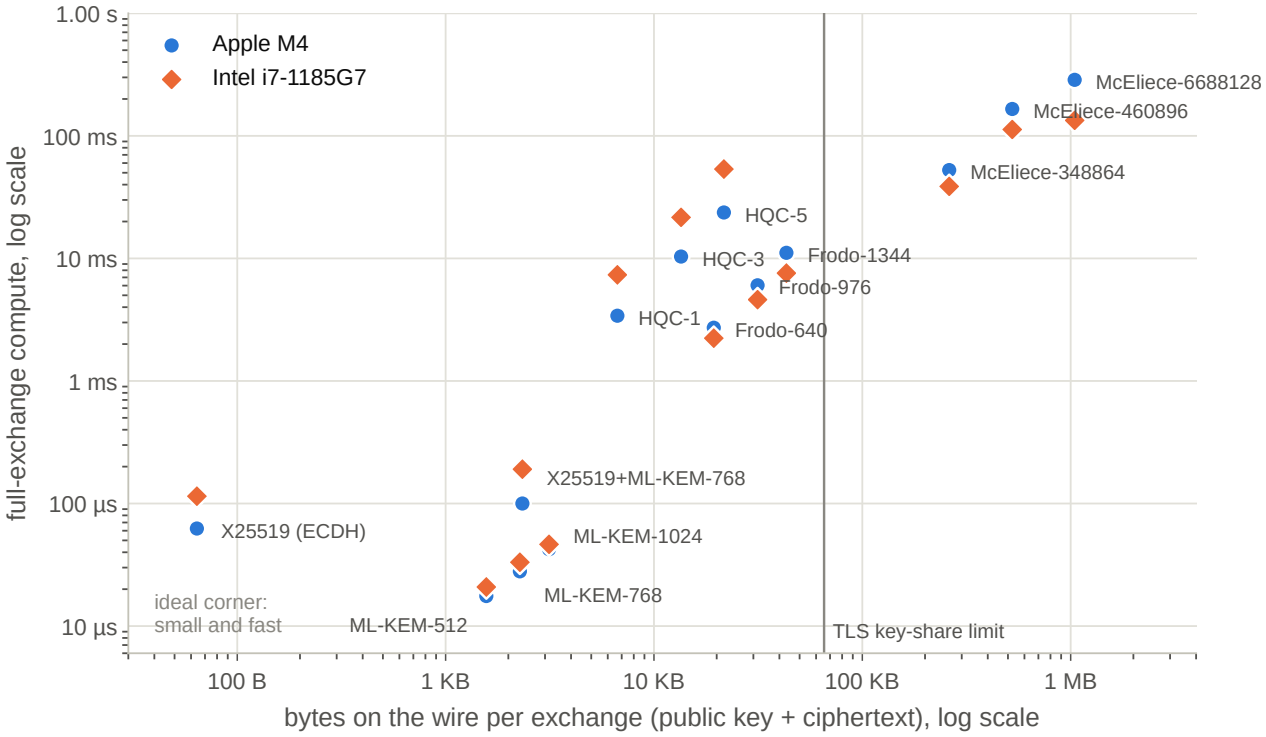


Figure 12. Compute vs. bytes on the wire, per full exchange. Bottom-left is best.

Takeaways

- **ML-KEM is the only KEM in the "small and fast" corner**; X25519 is smaller but slower.
- FrodoKEM and HQC cost both more bytes and more compute; McEliece trades huge keys for its tiny ciphertext.
- The chart is the core result of RQ1: the conservative choices are expensive *in both dimensions*.

6.5 Security level scaling

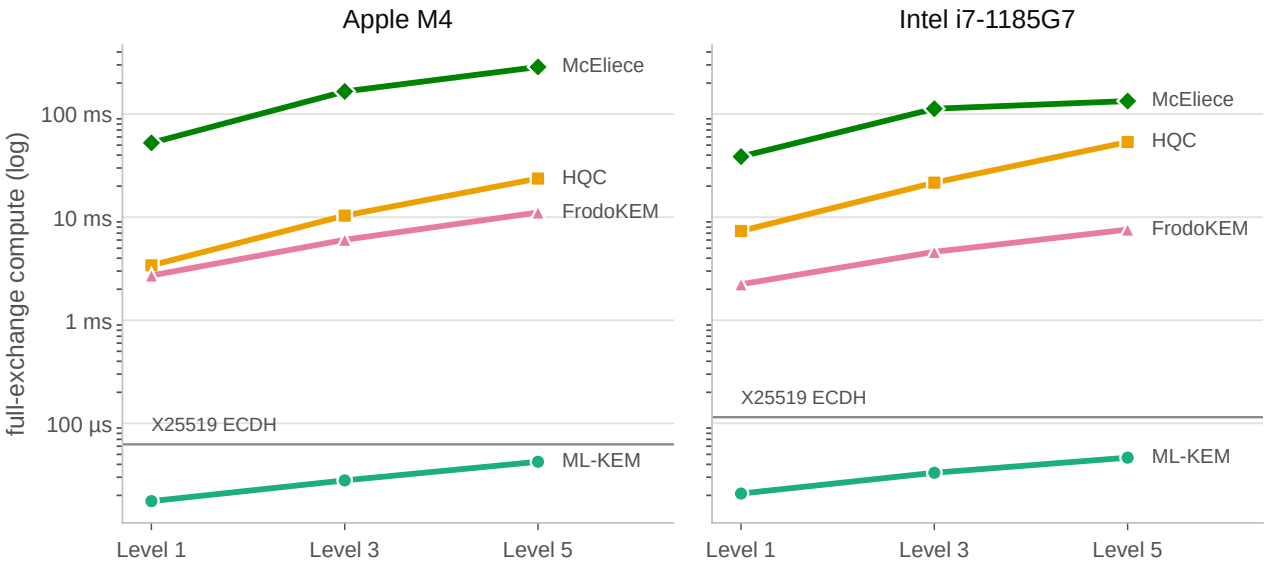


Figure 13. Full-exchange compute from NIST level 1 to 5 (log scale). ML-KEM stays below X25519 even at level 5.

- From level 1 to 5: ML-KEM ×2.2–2.4, HQC ×7, FrodoKEM ×3.4–4.1, McEliece ×3.5–5.4.

- **Why ML-KEM stays cheap:** its arithmetic is on 12-bit numbers (mod 3329), 8–16 per SIMD instruction, multiplied with the NTT. X25519 needs about 2,500 sequential 255-bit multiplications per operation (58k cycles per key pair on the M4). Large keys do not imply heavy computation.

6.6 Optimised vs. portable code

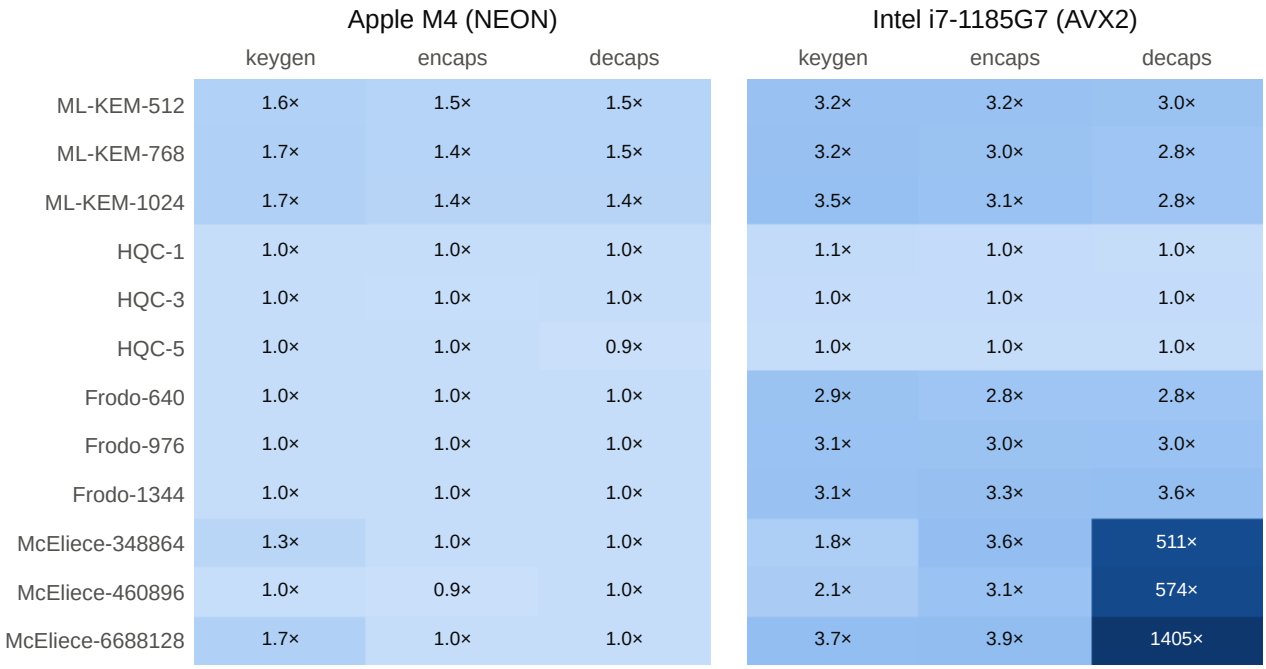


Figure 14. Speed-up of the optimised build over the portable one.

- **AVX2 (i7):** ML-KEM and FrodoKEM about 3×; McEliece decaps **510–1,400**× (bitsliced decoding instead of portable C).
- **NEON (M4):** only ML-KEM has NEON code, 1.4–1.7×; everything else is 1.0× (Table 4). The McEliece keygen values (0.95–1.7×) are noise from its retry loop.
- **HQC:** 1.0× on both, since liboqs 0.16 ships no AVX2 HQC. The HQC team's own AVX2 code (76k/150k/353k cycles at level 1 in its specification) is 33–49× faster than what we measured, so HQC's large gap to ML-KEM is partly an implementation gap.

6.7 ARM64 vs. x86-64

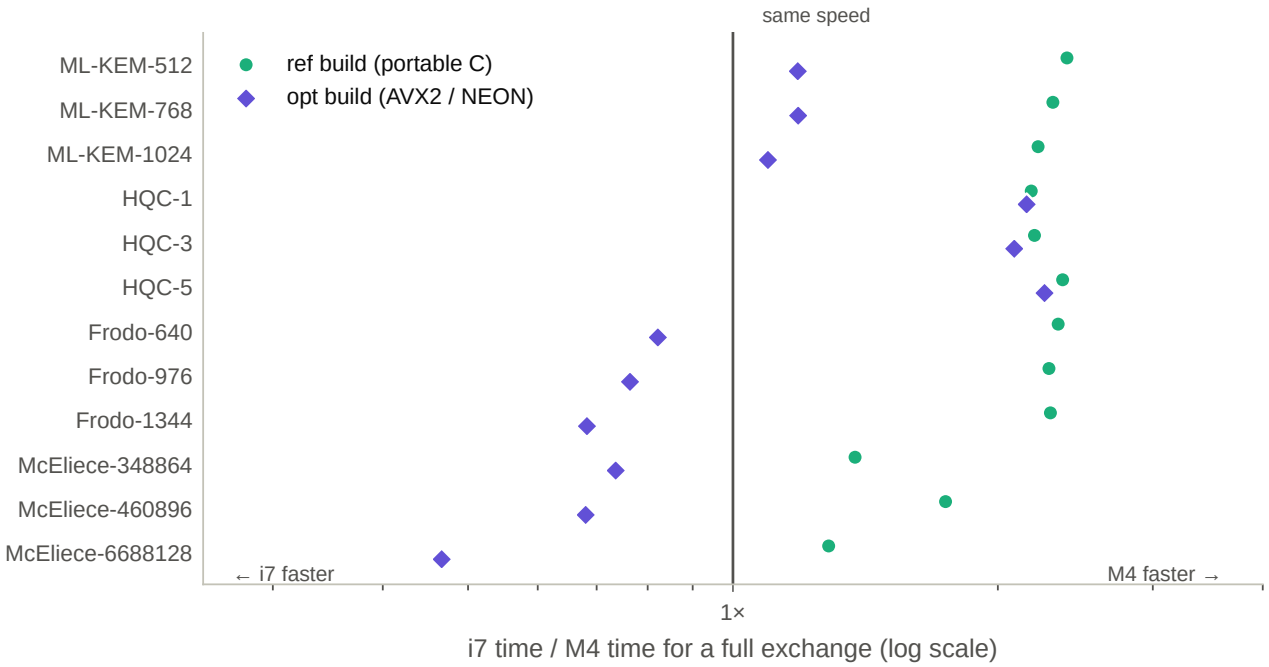


Figure 15. i7 time divided by M4 time for a full exchange. Right of 1×: the M4 is faster.

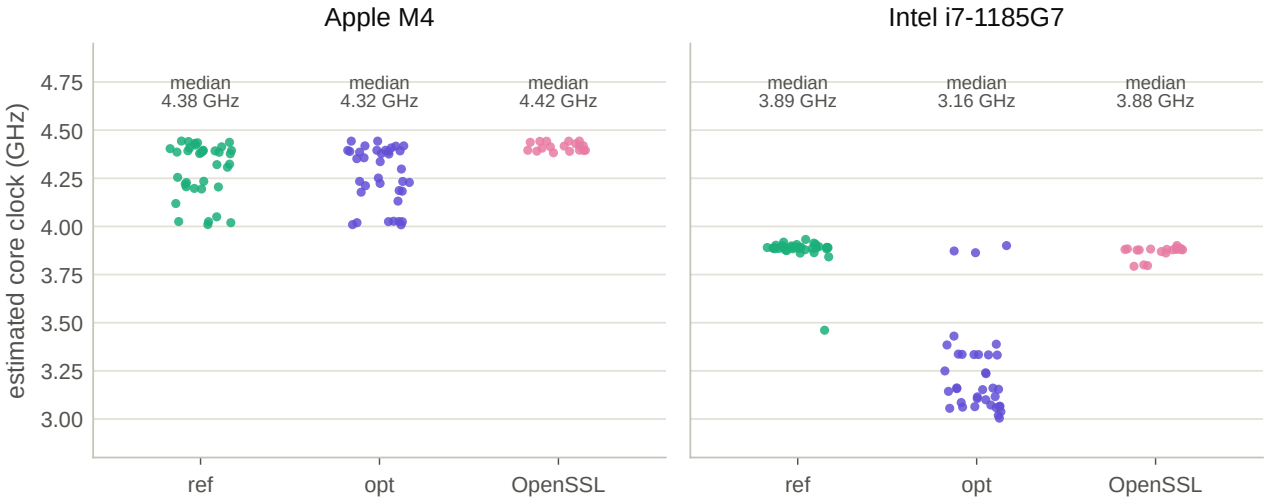


Figure 16. Core clock estimated around every timed loop. The i7 drops to ∼3.2 GHz while running AVX2 code; the M4 stays at 4.0–4.44 GHz (the lowest points are the longest loops, where the fanless Air throttles slightly).

Takeaways

- **On the same portable code the M4 is about 2.2× faster** (wider core, higher clock).
- **With optimised code the winner depends on what exists:** ML-KEM about equal (i7 1.1–1.2× slower), FrodoKEM and McEliece faster on the i7 (AVX2 code, no NEON code).
- **The i7 lowers its clock for AVX2 code** (3.9 → 3.2 GHz). In core cycles AVX2 ML-KEM-768 is 3.3–3.6× cheaper than portable (published for Kyber: 3.5–5.2×); in time the gain shrinks to 2.8–3.2×.

6.8 The classical baseline and the hybrid

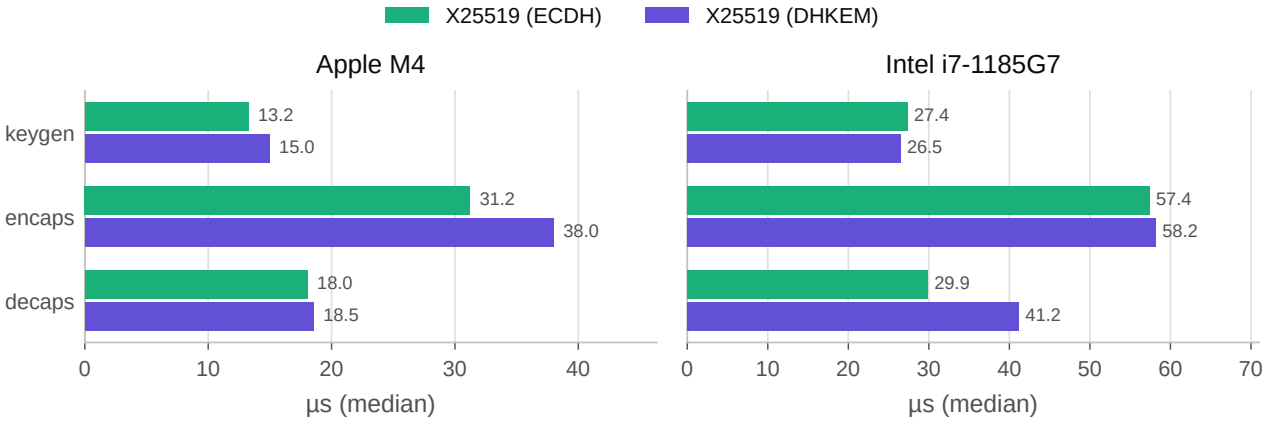


Figure 17. Plain X25519 ECDH vs. the same curve wrapped as a KEM (DHKEM, RFC 9180).

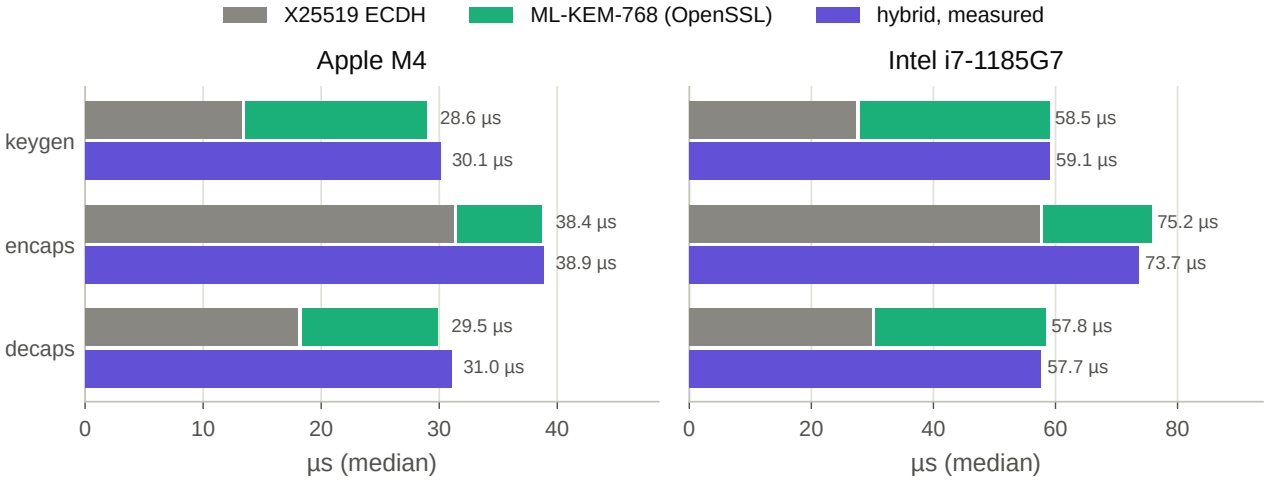


Figure 18. Measured hybrid vs. the sum of its parts (X25519 ECDH + OpenSSL's ML-KEM-768).

- The **DHKEM wrapper adds 10–14%** to a full exchange (one HKDF per side), so plain ECDH is the fairer baseline. The i7's DHKEM decaps row is noisy (IQR 28%).
- **The hybrid costs its two parts added together** (within 5%): combining them is essentially free.
- OpenSSL's ML-KEM keygen is 1.8–3.2× slower than liboqs `opt`, which matters for the hybrid's client side.

6.9 Memory: stack use

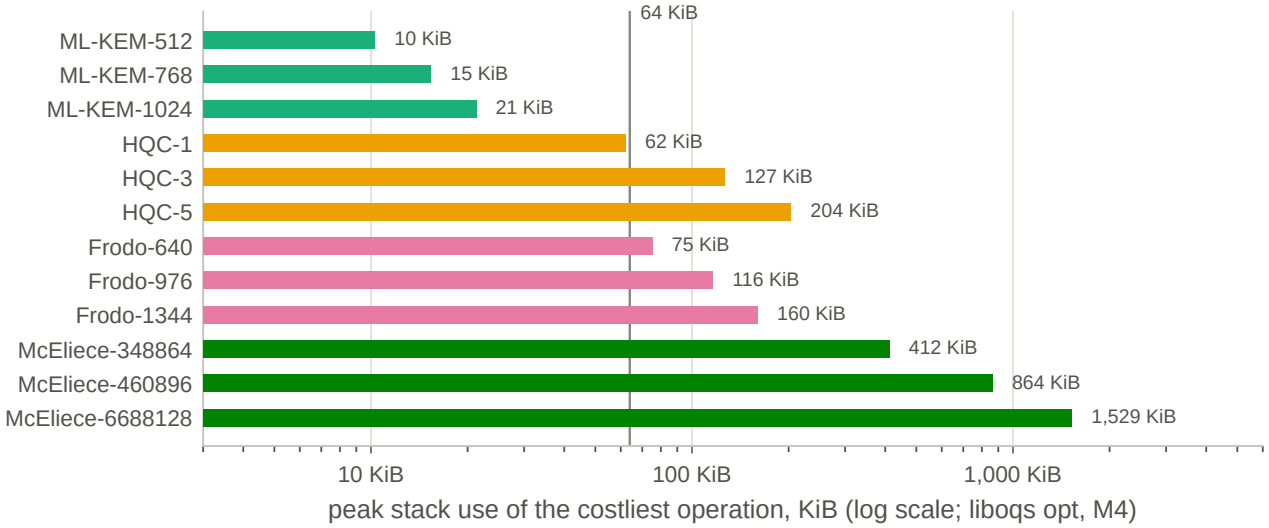


Figure 19. Peak stack of the most demanding operation (liboqs opt, M4).

ML-KEM fits in 10–21 KiB. HQC and FrodoKEM need 60–200 KiB, and McEliece 0.4–1.5 MiB, which rules it out of small embedded devices and constrained threads.

6.10 Validation against published results

	keygen	encaps	decaps	ours/pub.	published on
ML-KEM-768, AVX2	33k / 53k	34k / 68k	44k / 53k	0.65	Kyber R3 spec, Haswell
ML-KEM-768, portable	119k / 199k	120k / 235k	144k / 275k	0.54	Kyber R3 spec, Haswell
HQC-1, portable	3.6M / 4.6M	7.3M / 9.1M	11.5M / 13.9M	0.81	HQC spec 2025, i7-11850H
Frodo-640, AVX2	2.1M / 4.0M	2.4M / 4.2M	2.4M / 4.1M	0.56	FrodoKEM spec, Skylake
McEliece-348864, AVX2	120.0M / 46.5M	41k / 44k	115k / 134k	1.46	McEliece R3, Haswell

Table 8. Our i7 core cycles (estimated) vs. the designers' published cycle counts ("ours / published"). Published values use turbo off on older Intel cores, and the Kyber numbers are for the Round-3 code, the pre-standard version of ML-KEM.

- **Agreement where the code is the same:** HQC (same portable code, same Intel generation) within 20%; McEliece encaps/decaps within 7–14%.
- **Fewer cycles for ML-KEM and FrodoKEM:** newer, faster implementations (mlkem-native in liboqs; 4-way SHAKE) on a newer core.
- **Open discrepancy:** McEliece-348864 keygen at 120M cycles is 2.6× the published 46.5M, with 100 samples. To be investigated.

7 Findings, limitations and next steps

Key findings

1. All implementations are correct on both architectures (E4), including against NIST's FIPS 203 vectors.
2. **ML-KEM is cheaper than classical X25519** in compute (2.2–3.5× at level 3); its cost is bandwidth (35× more bytes), not CPU.
3. The deployed hybrid costs 1.6–1.7× X25519, exactly its parts added together.
4. Conservative alternatives cost 1–3 orders of magnitude more compute *and* more bytes: HQC 55–64×, FrodoKEM 19–43×, McEliece 337–841× at level 1.
5. **Implementation matters as much as the algorithm:** AVX2 changes McEliece decaps by up to 1,400×, missing NEON code makes the M4 slower than an older i7, and HQC lacks optimised code in liboqs.

Limitations

- Both machines run Linux in a VM (Docker Desktop, WSL2); per-call overhead is small for these CPU-bound loops but not zero.
- The ARM timer ticks every 41.7 ns, i.e. 0.4–0.8% of the fastest operations (5–10 μs); the median of 1,000 calls averages this out.
- The i7's tick-to-ns conversion differed by 2.4% between the main run and the appended X25519-ECDH rows (WSL2 clock); core-cycle values are unaffected.
- McEliece keygen timing varies a lot by nature (IQR 55–120%).
- One laptop per architecture; results describe these CPUs, not every ARM or x86 chip.

Next (Part 2): E2 measures what these costs mean inside real TLS 1.3 handshakes under network delay and loss; E3 tests the implementations for timing side channels (dudect, Valgrind, KyberSlash).

A Full E1 results

KEM	build	keygen	encaps	decaps	kg cyc	enc cyc	dec cyc	IQR
X25519 (ECDH)	openssl	13.2 μs	31.2 μs	18 μs	58k	138k	80k	4%
X25519+ML-KEM-768	openssl	30.1 μs	38.9 μs	31 μs	133k	172k	138k	8%
X25519 (DHKEM)	openssl	15 μs	38 μs	18.5 μs	66k	167k	82k	9%
ML-KEM-512	ref	8.54 μs	8.33 μs	9.96 μs	37k	36k	42k	2%
ML-KEM-512	opt	5.21 μs	5.67 μs	6.71 μs	23k	25k	29k	9%
ML-KEM-512	openssl	9.62 μs	5.33 μs	8.79 μs	42k	23k	39k	9%
ML-KEM-768	ref	14.5 μs	13 μs	15.2 μs	64k	57k	67k	8%
ML-KEM-768	opt	8.5 μs	8.96 μs	10.5 μs	37k	39k	46k	9%
ML-KEM-768	openssl	15.3 μs	7.17 μs	11.5 μs	67k	32k	51k	10%
ML-KEM-1024	ref	22.4 μs	19.7 μs	22.5 μs	98k	86k	99k	8%
ML-KEM-1024	opt	12.8 μs	13.7 μs	15.9 μs	57k	60k	69k	1%
ML-KEM-1024	openssl	23.2 μs	9.33 μs	15.2 μs	102k	41k	67k	10%
HQC-1	ref	561 μs	1.12 ms	1.71 ms	2,490k	4,996k	7,548k	1%
HQC-1	opt	561 μs	1.12 ms	1.72 ms	2,492k	4,929k	7,208k	1%
HQC-3	ref	1.69 ms	3.36 ms	5.09 ms	7,475k	14.8M	21.4M	1%
HQC-3	opt	1.69 ms	3.53 ms	5.14 ms	6,971k	14.9M	20.7M	1%
HQC-5	ref	3.68 ms	7.44 ms	11.2 ms	15.5M	29.9M	44.9M	1%
HQC-5	opt	3.72 ms	7.56 ms	12.4 ms	15.0M	30.3M	52.3M	1%
Frodo-640	ref	862 μs	923 μs	917 μs	3,804k	4,088k	3,883k	1%
Frodo-640	opt	873 μs	927 μs	918 μs	3,837k	4,038k	3,947k	1%
Frodo-976	ref	1.93 ms	2.09 ms	2.07 ms	8,594k	8,821k	8,403k	1%
Frodo-976	opt	1.91 ms	2.07 ms	2.05 ms	8,391k	8,285k	8,269k	1%
Frodo-1344	ref	3.49 ms	3.81 ms	3.72 ms	14.6M	15.7M	15.0M	1%
Frodo-1344	opt	3.57 ms	3.79 ms	3.75 ms	14.9M	15.3M	15.7M	1%
McEliece-348864	ref	51.8 ms	32.5 μs	13.6 ms	228.1M	140k	60.0M	72%
McEliece-348864	opt	38.6 ms	32.6 μs	13.9 ms	163.2M	141k	61.4M	65%
McEliece-460896	ref	134 ms	66.2 μs	24.6 ms	566.0M	290k	107.9M	94%
McEliece-460896	opt	141 ms	73.8 μs	24.7 ms	596.6M	326k	109.9M	68%
McEliece-6688128	ref	413 ms	121 μs	47.6 ms	1,737.4M	484k	205.2M	119%
McEliece-6688128	opt	239 ms	121 μs	47.7 ms	1,009.6M	486k	202.7M	123%

Table 9. Apple M4: medians per call, estimated core cycles, and the largest relative IQR of the three operations.

KEM	build	keygen	encaps	decaps	kg cyc	enc cyc	dec cyc	IQR
X25519 (ECDH)	openssl	27.4 µs	57.4 µs	29.9 µs	104k	218k	114k	1%
X25519+ML-KEM-768	openssl	59.1 µs	73.7 µs	57.7 µs	231k	286k	224k	1%
X25519 (DHKEM)	openssl	26.5 µs	58.2 µs	41.2 µs	103k	226k	159k	28%
ML-KEM-512	ref	18.5 µs	20.7 µs	25.1 µs	72k	80k	98k	1%
ML-KEM-512	opt	5.85 µs	6.52 µs	8.46 µs	20k	22k	28k	1%
ML-KEM-512	openssl	21.1 µs	13.7 µs	21.2 µs	82k	53k	82k	1%
ML-KEM-768	ref	30.6 µs	30.9 µs	37.1 µs	119k	120k	144k	1%
ML-KEM-768	opt	9.6 µs	10.3 µs	13.2 µs	33k	34k	44k	0%
ML-KEM-768	openssl	31.1 µs	17.8 µs	27.9 µs	121k	69k	108k	1%
ML-KEM-1024	ref	46.1 µs	44.7 µs	52.6 µs	179k	174k	205k	1%
ML-KEM-1024	opt	13.3 µs	14.6 µs	18.6 µs	44k	47k	60k	1%
ML-KEM-1024	openssl	47.4 µs	23.6 µs	36.8 µs	184k	92k	143k	2%
HQC-1	ref	1.22 ms	2.45 ms	3.73 ms	4,757k	9,485k	14.5M	1%
HQC-1	opt	1.16 ms	2.36 ms	3.83 ms	3,642k	7,320k	11.5M	3%
HQC-3	ref	3.72 ms	7.41 ms	11.2 ms	14.5M	28.8M	43.7M	1%
HQC-3	opt	3.58 ms	7.17 ms	10.9 ms	11.2M	22.6M	33.3M	3%
HQC-5	ref	8.79 ms	17.5 ms	26.5 ms	34.2M	68.5M	103.0M	0%
HQC-5	opt	8.8 ms	18.1 ms	26.7 ms	27.0M	54.6M	81.7M	3%
Frodo-640	ref	2.01 ms	2.16 ms	2.15 ms	7,830k	8,422k	8,377k	1%
Frodo-640	opt	681 µs	785 µs	767 µs	2,148k	2,423k	2,418k	4%
Frodo-976	ref	4.45 ms	4.77 ms	4.73 ms	17.3M	18.6M	18.5M	1%
Frodo-976	opt	1.44 ms	1.6 ms	1.56 ms	4,559k	4,899k	4,792k	2%
Frodo-1344	ref	8.06 ms	8.65 ms	8.59 ms	31.3M	33.6M	33.8M	0%
Frodo-1344	opt	2.58 ms	2.59 ms	2.41 ms	8,604k	8,405k	8,164k	7%
McEliece-348864	ref	71.4 ms	46.8 µs	18.6 ms	277.9M	181k	72.0M	69%
McEliece-348864	opt	38.6 ms	13 µs	36.5 µs	120.0M	41k	115k	19%
McEliece-460896	ref	234 ms	90.5 µs	41.9 ms	918.7M	353k	161.1M	100%
McEliece-460896	opt	113 ms	29.3 µs	73.1 µs	345.3M	89k	224k	50%
McEliece-6688128	ref	496 ms	153 µs	95.6 ms	1,939.7M	593k	330.9M	86%
McEliece-6688128	opt	134 ms	39.4 µs	68 µs	521.2M	153k	263k	55%

Table 10. Intel i7-1185G7, same columns.

B Reproducing this report

Run on each machine (`make kat bench` for E4 and E1 only), then:

```
make all
python3 analysis/part1_figures.py
cd report/part1 && latexmk -pdf part1.tex
```

Inputs: newest `e1_*` and `e4_*` file per architecture in `results/raw`. Colours: machines are always blue (M4) and orange (i7); the palettes were checked for colour-vision deficiency, and every value is also given as a label or in a table.

Tools. Code, analysis scripts and this document were written with the help of Claude (Anthropic), an AI assistant; the experimental design, the measurements and the interpretation are the author's responsibility.

References

[1] NIST, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*, Aug 2024.
[2] NIST, *IR 8545: Status Report on the Fourth Round of the NIST PQC Standardization Process*, Mar 2025 (HQC selected).
[3] C. Aguilar Melchor et al., *HQC specifications*, 2025-08-22, pqc-hqc.org.
[4] E. Alkim et al., *FrodoKEM specification*, 2021-06-04, frodokem.org; ISO/IEC 18033-2:2006/Amd 2:2026.
[5] D. J. Bernstein et al., *Classic McEliece: conservative code-based cryptography*, NIST Round-3 submission, 2020; ISO standard 2026.
[6] R. Avanzi et al., *CRYSTALS-Kyber, algorithm specifications (Round 3)*, 2021.
[7] A. Langley, M. Hamburg, S. Turner, *RFC 7748: Elliptic Curves for Security*, 2016.
[8] R. Barnes et al., *RFC 9180: Hybrid Public Key Encryption (DHKEM)*, 2022.
[9] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, D. Stebila, *Post-quantum hybrid ECDHE-MLKEM key agreement for TLS 1.3*, IETF draft-ietf-tls-ecdhe-mlkem.
[10] Open Quantum Safe, *liboqs 0.16.0*, openquantumsafe.org; PQ Code Package, *mlkem-native*.
[11] H. Becker et al., *Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1*, TCHES 2022.
[12] BSI, *TR-02102-1: Cryptographic Mechanisms: Recommendations and Key Lengths*.